

Apuntes-segundo-parcial-BD.pdf



Luneko



Bases de Datos



2º Grado en Ingeniería Informática del Software



**Escuela de Ingeniería Informática
Universidad de Oviedo**

Máster

Online en Ciberseguridad

Nº1 en España según El Mundo



**Hasta el 46%
de beca**



Mejor Máster
según el
Ranking de
ELMUNDO

Para ser el mejor hay que aprender
de los mejores.

IMEF

Smart Education

Deloitte

Infórmate

Consigue Empleo o Prácticas

Matricúlate en IMF y accede sin coste a nuestro servicio de Desarrollo Profesional con más de 7.000 ofertas de empleo y prácticas al mes.



IMF
Smart Education

SEGUNDO PARCIAL BBDD

Álgebra relacional - Operaciones

Selección

El predicado en subíndice y la tabla entre paréntesis.

Para seleccionar las tuplas de la relación préstamo en que la sucursal es "Navacerrada" se escribe:

$$\sigma_{\text{nombre_sucursal} = \text{"Navacerrada"}}(\text{préstamo})$$

Admite igual, distinto, mayor, menor, mayor o igual, menor o igual en las comparaciones. Se pueden combinar varios predicados mediante **y**, **o** y **no**

Proyección

Operación unaria que devuelve su relación de argumentos, excluyendo algunos.

La consulta para crear una lista de todos los números e importes de los préstamos puede escribirse como

$$\Pi_{\text{número_préstamo}, \text{importe}}(\text{préstamo})$$

Composición de operaciones relacionales

Considérese la consulta más compleja "Buscar los clientes que viven en Peguerinos". Hay que escribir:

$$\Pi_{\text{nombre_cliente}}(\sigma_{\text{ciudad_cliente} = \text{"Peguerinos"}}(\text{cliente}))$$

Unión

Considérese una consulta para determinar el nombre de todos los clientes del banco que tienen una cuenta, un préstamo o ambas cosas.

$$\Pi_{\text{nombre_cliente}}(\text{prestatario}) \cup \Pi_{\text{nombre_cliente}}(\text{impositor})$$

Se eliminan valores duplicados.

Intersección

(...)

Diferencia

Todos los clientes del banco que tengan abierta una cuenta pero no tengan concedido ningún préstamo

¿Quieres conocer todos los servicios?



WUOLAH

$$\Pi_{\text{nombre_cliente}}(\text{impositor}) - \Pi_{\text{nombre_cliente}}(\text{prestatario})$$

Producto cartesiano

$$\sigma_{\text{nombre_sucursal} = \text{"Navacerrada"}}(\text{prestatario} \times \text{préstamo})$$

El producto cartesiano toma todos los emparejamientos posibles de cada tupla de prestatario con cada tupla de préstamo

Renombramiento

$$\Pi_{\text{cuenta.saldo}}(\sigma_{\text{cuenta.saldo} < d.saldo}(\text{cuenta} \times \rho_d(\text{cuenta})))$$

Esta expresión proporciona los saldos de la relación cuenta para los que aparece un saldo mayor en alguna parte de la relación cuenta (cuyo nombre se ha cambiado a d). El resultado contiene todos los saldos salvo el máximo.

Unión natural (natural join)

“Determinar el nombre de todos los clientes que tienen concedido un préstamo en el banco y averiguar su importe”.

$$\Pi_{\text{nombre_cliente}, \text{número_préstamo}, \text{importe}}(\text{prestatario} \bowtie \text{préstamo})$$

Hace un producto cartesiano, realiza una selección forzando la igualdad de los atributos comunes a ambos esquemas de relación y elimina los duplicados.

Restricciones de Integridad

Garantizan que las modificaciones realizadas en la base de datos por los usuarios autorizados no den lugar a una pérdida de la consistencia de los datos. Por tanto, protegen contra daños accidentales a las bases de datos.

Restricciones sobre una sola relación

Además de la restricción de “clave primario” encontramos las siguientes

- **Restricción not null**

El valor null es miembro de todos los dominios y, por tanto, un valor legal para todos los atributos de SQL. En casos en los que no se considere adecuado que un valor tome valor nulo se puede usar esta restricción para excluir los valores null del dominio. SQL prohíbe los valores nulos en la clave primario de los esquemas de las relaciones, por lo que no hace falta declararlos como not null.

La especificación not null se puede aplicar también a declaraciones de dominio definidas por los usuarios, por ejemplo:

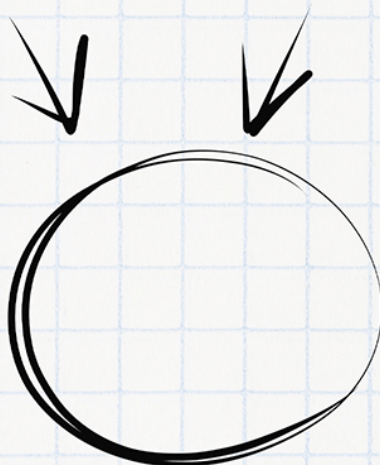
```
create domain Euros numeric(12,2) not null
```


Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

Bases de Datos



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR



- **Restricción unique**

unique (Aj1,Aj2, ... ,Ajm)

Unique indica que los atributos forman una clave candidata, es decir, ningún par de tuplas de la relación puede ser igual en todos los atributos de la clave primaria. Se permite que los atributos de la clave candidata tengan valores nulos, a menos que se haya especificado de manera explícita.

- **Cláusula check**

Puede aplicarse a declaraciones de relaciones y dominios. Cuando se aplica a relaciones, la cláusula **check(P)** especifica un predicado P que deben cumplir todas las tuplas de la relación.

Se usa para garantizar que los valores de los atributos cumplan las condiciones especificadas.

Cuando se aplica a un dominio, permite a los diseñadores de esquemas especificar un predicado que debe cumplir cualquier valor asignado a las variables cuyo tipo de datos sea el dominio. Por ejemplo:

```
create domain SalarioPorHora numeric(5,2)
constraint prueba_valor_salario check(value >= 6.00)
```

Se permiten subconsultas que hagan referencias a otras relaciones en la condición.

- **Integridad referencial**

Consiste en una condición que garantiza que el valor que aparece en una relación para un conjunto dado de atributos aparezca también para un conjunto determinado de atributos en otra relación.

Las claves externas pueden especificarse mediante la cláusula **foreign key**.

De manera predeterminada, en SQL las claves externas hacen referencia a los atributos de la clave primaria de la tabla referenciada. SQL también soporta una versión de la cláusula **references** en la que se puede especificar una lista de atributos de la relación a la que se hace referencia. Dicha lista debe declararse como clave candidata de la relación a la que hace referencia.

Cuando se viola una restricción de integridad referencial, se suele rechazar la acción que ha causado esa violación. Sin embargo, la cláusula **foreign key** puede especificar que el sistema modifique la tupla de la relación que hace la referencia para restaurar la restricción.

```
create table cuenta
( ...
foreign key (nombre_sucursal) references sucursal
on delete cascade
on update cascade,
...)
```

La cláusula **cascade** hace que, al borrar una tupla de datos de sucursal y producirse una violación de la restricción, se borre también la tupla de datos a la que estaba asociada la cuenta, lo mismo pasa con la actualización, cambiando el valor del campo *nombre_sucursal* de las tuplas de cuenta. Pueden especificarse también **otras acciones** diferentes en lugar de **cascade** (set null o set default).

Las cadenas de dependencias de clave externa que afectan a varias relaciones pueden hacer que un borrado o actualización se propague por todas ellas. En el caso de que una

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

violación no pueda resolverse por cascada, el sistema aborta la transacción y todas las modificaciones se deshacen.

Los nulls complican la semántica de este tipo de restricciones. Se permite que los atributos de las claves externas sean nulos, siempre y cuando no se emplee not null. Si todas las columnas de una clave externa son no nulas en una tupla dada, se utiliza la definición habitual de las restricciones de clave externa. En caso contrario, la tupla se define de manera automática para satisfacer la restricción.

SQL ofrece constructores para modificar el comportamiento en valores nulos.

Se pueden añadir restricciones de integridad a relaciones ya existentes utilizando:

`alter table <nombre tabla> add <restriccion>`

Al usar este comando, el sistema comprueba que la relación satisfaga la restricción y, si se cumple, se añade la restricción a la relación, o se rechaza en caso contrario.

Las transacciones pueden constar de varios pasos, y las restricciones de integridad pueden violarse temporalmente en un paso y arreglarse en el siguiente. Para tratar estas situaciones se permite añadir la cláusula **initially deferred** a la restricción, la cual pasa a comprobarse al final de la transacción, en vez de los pasos intermedios. Las restricciones pueden especificarse como **deferrable**, haciendo que se compruebe inmediatamente por predeterminado, pero puede diferirse si se desea. Para aquellas restricciones diferibles, la ejecución de la instrucción **set constraints <lista-restricciones> deferred** como parte de una transacción hace que comprueben las restricciones al final.

Predeterminado: Las restricciones se comprueban inmediatamente y muchas implementaciones no soportan la comprobación diferida.

• Asertos

Predicados que expresan una condición que la **base de datos debe satisfacer siempre**.

Las restricciones de dominio y las de integridad referencial son formas especiales de asertos, siendo fáciles de verificar y pueden afectar a una gran variedad de aplicaciones de bases de datos, pero no pueden emplearse para todo.

`create assertion <nombre-aserto> check <predicado>`

Cuando se crea un aserto, el sistema comprueba su validez, si es válido, sólo se permiten modificaciones que lo cumplan. Esta comprobación puede producir mucha sobrecarga si los asertos son complejos.

Diseño de BBDD Relacionales

7.3 DESCOMPOSICIÓN MEDIANTE DEPENDENCIAS FUNCIONALES

7.3.2 Forma normal de Boyce - Codd

Elimina todas las **redundancias** que se pueden descubrir a partir de las dependencias funcionales, pero puede que queden otros tipos de redundancias.

El esquema de relación R está en la **FNBC** respecto al conjunto F de dependencias funcionales si, para todas las dependencias funcionales de F^+ de la forma $\alpha \rightarrow \beta$ donde

$\alpha \subseteq R$ y $\beta \subseteq R$, se cumple al menos una de las **condiciones**:

- $\alpha \rightarrow \beta$ es una dependencia funcional trivial, es decir, $\beta \subseteq \alpha$
- α es superclave del esquema R

WUOLAH

Los diseños de bases de datos están en FNBC si cada miembro del conjunto de esquemas que constituye el diseño está en FNBC.

El esquema *prestatario_préstamo* = (*id_cliente*, *número_préstamo*, *importe*) no se haya en FNBC, la dependencia *número_préstamo* → *importe* se cumple en *prestatario_préstamo*, pero *número_préstamo* no es superclave

Descomponer esquemas que no se hallen en FNBC:

Sea R uno, queda, al menos, una dependencia funcional no trivial $\alpha \rightarrow \beta$ tal que α no es superclave de R, en el diseño se **sustituye** R por dos esquemas:

- ($\alpha \cup \beta$)
- ($R - (\beta - \alpha)$)

En el caso anterior de *prestatario_préstamo*, $\alpha = \text{número_préstamo}$, $\beta = \text{importe}$ y *prestatario_préstamo* se sustituye por:

- ($\alpha \cup \beta$) = (*número_préstamo*, *importe*)
- ($R - (\beta - \alpha)$) = (*id_cliente*, *número_préstamo*) → $\beta - \alpha = \beta$

Cuando se descomponen esquemas que no se hallan en FNBC, puede que algunos de los esquemas resultantes no se hallen en FNBC, por lo que habrá que hacer más descomposiciones hasta que el resultado final sea un **conjunto de esquemas FNBC**.

7.3.3 FNBC y la conservación de las dependencias

Hay varias maneras de expresar las **restricciones de consistencia** de las BBDD, pero la comprobación de éstas puede ser **costosa**, por lo que resulta útil diseñar la BBDD de manera que las restricciones se puedan comprobar eficientemente considerando sólo una relación.

La **descomposición** de la FNBC puede impedir la comprobación eficiente de algunas dependencias funcionales.

Lo ideal es que haya un **solo conjunto de relaciones** al que poder hacer referencia para que se cumpla una restricción

Cuando un diseño hace computacionalmente difícil que se cumpla una dependencia funcional, este diseño no **conserva las dependencias**

La conservación de dependencias suele considerarse deseable, por lo que se considera otra forma normal, más **débil** que la FNBC que permita que se conserven las dependencias (3FN)

7.3.4 Tercera forma normal

3NF relaja ligeramente la restricción de que todas las dependencias no triviales sean de la forma $\alpha \rightarrow \beta$ donde α es una superclave, **permitiendo dependencias funcionales no triviales** cuya parte izquierda no sea una superclave.

Claves candidatas: superclaves mínimas → superclaves en las que ningún subconjunto propio sea también superclave.

El esquema de relación R está en **3FN** respecto a un conjunto F de dependencias funcionales si, para todas las dependencias funcionales de F^+ de la forma $\alpha \rightarrow \beta$ donde $\alpha \subseteq R$ y $\beta \subseteq R$, se cumple al menos una de las **condiciones**:

- $\alpha \rightarrow \beta$ es una dependencia funcional trivial
- α es superclave de R
- Cada atributo A de $\beta - \alpha$ está **contenido** en alguna clave candidata de R

La tercera condición es la única diferente a las de FNBC, pero hay que tener en cuenta que una sola clave candidata no tiene por qué contener todos los atributos de $\beta - \infty$, sino que cada atributo A de $\beta - \infty$ puede estar contenido en una clave candidata diferente.

Ésta representa, en cierto sentido, una **relajación mínima de las condiciones** de la FNBC, ayudando a garantizar que cada esquema tenga una descomposición que conserve las dependencias en la 3FN.

Cualquier esquema que satisfaga la FNBC satisface también la 3FN, ya que cada una de sus dependencias funcionales satisfará una de las dos primeras condiciones.

Cuando no hay ningún diseño FNBC que conserve las dependencias, hay que guardar **equilibrio** entre la FNBC y la 3FN.

7.3.5 Formas normales superiores

En algunos casos el empleo de las dependencias funcionales para la descomposición de los esquemas puede no ser suficiente para evitar la **repetición** innecesaria de información y, por ello, se han definido otras dependencias y formas normales.